THE OPEN SOURCE AUDIT

A Capstone Project Report

Course: Open Source Software | Units 1–5 | Max Marks: 100

Software Chosen:

Linux Kernel — Operating System Foundation | Licence: GPL v2

Student Name	Priyanshu
Registration No.	24BCY10108
Slot	A-24
Date of Submission	29-03-2026

1. Introduction

This report is a structured audit of the Linux Kernel, the most influential piece of open-source software ever written. The Linux Kernel is the core of the Linux operating system — the layer of software that sits directly above computer hardware and manages every other process running on a machine. It handles memory, files, processes, networking, and device drivers. Without the kernel, no application can run, no network packet can travel, and no file can be saved.

What makes the Linux Kernel remarkable is neither its technical complexity nor its scale, though both are extraordinary. What makes it remarkable is the story of how it was built: by a single student who could not afford a Unix system, shared freely with the world under a licence that ensured it would never be taken private, and then grown over three decades by tens of thousands of contributors into the foundation on which the modern internet, the cloud, and nearly every supercomputer in the world runs.

The Linux Kernel runs inside Android phones, Tesla vehicles, the International Space Station, the world's top 500 supercomputers, and the servers that power Google, Amazon, Meta, and Netflix. It is the single most deployed piece of software in human history — and every line of its source code is publicly readable, auditable, and free to modify under the terms of the GNU General Public Licence version 2 (GPL v2).

This report examines the Linux Kernel as an open-source artefact across four structured parts, covering Units 1–5 of the Open Source Software course. Five shell scripts developed and tested on Ubuntu Linux accompany this report in the GitHub repository.

Part A — Origin and Philosophy (Units 1 & 2)

A1. The Problem the Linux Kernel Was Created to Solve

To understand why the Linux Kernel exists, one must understand the world of operating systems in 1991. At that time, Unix was the dominant operating system for serious computing. Developed at Bell Labs in the late 1960s and refined over two decades, Unix was stable, powerful, and well-designed. It was the system on which nearly all professional software development, scientific computing, and university education took place. The problem was that Unix was proprietary and expensive. Source code was locked behind licences that cost thousands of dollars. Students and individual researchers could not legally access it.

There were alternatives. Richard Stallman had launched the GNU Project in 1983 with the explicit goal of creating a completely free Unix-like operating system. By 1991, the GNU Project had produced a remarkable collection of tools — a compiler (GCC), a text editor (Emacs), shell utilities, and more. But the GNU Project was missing its most essential component: a kernel. The GNU Hurd kernel was under development but years away from being usable. Minix, an academic Unix clone written by Andrew Tanenbaum for teaching operating system concepts, existed and ran on personal computers — but its licence restricted redistribution and modification.

Linus Benedict Torvalds was a 21-year-old computer science student at the University of Helsinki. He had acquired a personal computer — an IBM-compatible 386 machine — and wanted to run Unix on it to continue the kind of work he did on the university's Sun workstations. He could not afford a commercial Unix licence. He began using Minix, but quickly grew frustrated with its limitations: it could not be modified freely, it had no proper terminal emulation, and Tanenbaum had stated publicly that he had no intention of making it a real operating system.

On 25 August 1991, Torvalds posted a now-famous message to the comp.os.minix Usenet newsgroup: he was working on a free operating system — just a hobby, he said, and probably nothing big or professional like GNU. He called it Linux, combining his first name with the Unix suffix. What made Linux different from the outset was not technical superiority but a philosophical commitment: Torvalds released it under a licence that allowed anyone to read the code, modify it, and redistribute it. The first release, version 0.01, contained 10,239 lines of code. Linux 6.x, released in 2023, contains over 36 million lines.

The core problem Linux solved was access. It gave every student, researcher, and developer in the world a real, modifiable, extensible Unix-like kernel that ran on affordable hardware. It completed the GNU system and together they became GNU/Linux — for the first time in computing history, a complete, professional-grade operating system was freely available to anyone with a computer and an internet connection.

A2. The Licence — What GPL v2 Actually Says

The Linux Kernel is licensed under the GNU General Public Licence version 2 (GPL v2). This is one of the most significant licences in the history of software — not merely because of the projects it governs, but because of the philosophy it encodes.

The Four Freedoms

The Free Software Foundation defines four essential freedoms that any truly free software licence must grant:

- Freedom 0: The freedom to run the programme for any purpose.
- Freedom 1: The freedom to study how the programme works and change it. Access to the source code is a precondition.
- Freedom 2: The freedom to redistribute copies so you can help others.
- Freedom 3: The freedom to distribute copies of your modified versions to others.

GPL v2 grants all four of these freedoms explicitly and unconditionally. The Linux Kernel's source code is publicly available on kernel.org. Anyone can download every line of it, read it, compile it, modify it, and run it.

The Copyleft Condition — Can a Company Modify Linux and Keep It Proprietary?

GPL v2 gives a clear and deliberate answer: no. GPL v2 is a strong copyleft licence. If you distribute a modified version of the Linux Kernel, you must release the source code of your modifications under the same GPL v2 terms. You cannot take the Linux Kernel, add proprietary improvements, and ship a product without sharing your changes. This is the reason Android manufacturers are required to publish the kernel source for their devices and why the GPL v2 has been called a 'freedom-preserving' licence.

GPL v2 versus MIT Licensing

MIT is a permissive licence: it grants all four freedoms but imposes no conditions on distribution. A company can take MIT-licensed code, make proprietary modifications, and ship a closed-source product. GPL v2, by contrast, enforces reciprocity: you received this software freely, and if you give it to others, you must give them the same freedoms. For a project like the Linux Kernel, GPL v2's copyleft provision has prevented the fragmentation and proprietary enclosure that destroyed Unix. If I were building infrastructure that society depends on, I would choose GPL v2. If building a widely-adopted library, I would choose MIT.

Free as in Beer versus Free as in Freedom

Richard Stallman's distinction between 'free as in free beer' (gratis) and 'free as in freedom' (libre) is essential here. The kernel is free in both senses — downloadable at no cost and GPL v2-protected. Android illustrates the complexity: manufacturers must publish the kernel source (free as in freedom at the kernel level), but proprietary drivers and Google Play Services above it are neither free as in beer nor free as in freedom for the user.

A3. The Ethics of Open Source

Should All Software Be Open Source?

The Linux Kernel's history makes this question concrete. The kernel runs inside medical devices, aircraft navigation systems, and financial exchanges. For software operating in the public interest at this scale, the argument for mandatory openness is compelling: when society depends on a system, society should be able to audit it. A proprietary kernel in a pacemaker is an accountability gap that patients cannot fix.

The counter-argument is equally real. The kernel was created during Torvalds' unpaid holiday time and sustained by a combination of volunteer passion and corporate investment. Not all software has this dynamic. The more defensible position is graduated: software operating in the

public interest carries a stronger ethical obligation toward openness than private commercial applications.

Is It Ethical to Profit from Free Community Labour?

Red Hat, IBM, Google, Amazon, and Microsoft have collectively derived hundreds of billions in value from the Linux Kernel. In return, they contribute engineering time — but the relationship is asymmetric. Individual maintainers who carry enormous responsibility often do so on volunteer time. The Linux Foundation represents the ecosystem's attempt to address this: it employs full-time kernel developers and manages infrastructure. The ethical failure is not that companies profit from Linux; it is that this realisation has been slow to translate into proportional support for the individuals whose sustained effort makes it possible.

Standing on the Shoulders of Giants

Newton's phrase captures something important about the Linux Kernel that proprietary software philosophy cannot match. The kernel was built using GCC, incorporated ideas from BSD Unix, Minix, and decades of openly published academic research. The ability to read predecessors, understand them, and improve on them is the mechanism by which computing knowledge accumulates across generations. Proprietary software breaks this chain: when a company discontinues a product, the knowledge embedded in its code disappears behind legal barriers. Open source ensures that knowledge is never walled off, and that every innovation built on Linux becomes part of the foundation for the next generation.

Part B — Linux Footprint (Unit 2)

B1. Installation Method

The Linux Kernel can be obtained in two fundamentally different ways. On most distributions, the kernel is managed via the distribution's package manager. On Ubuntu/Debian-based systems:

```
sudo apt update && sudo apt install linux-image-$(uname -r)
```

For custom configuration or kernel development, compilation from source is required:

```
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.6.tar.xz
tar -xvf linux-6.6.tar.xz && cd linux-6.6
make menuconfig    # Interactive configuration
make -j$(nproc)    # Compile using all available CPU cores
sudo make modules_install && sudo make install
```

B2. Key Directories

Path	Contents	Purpose
/boot	vmlinuz, initrd.img, System.map	Kernel image and boot config
/lib/modules/$(uname -r)	drivers/, kernel/, modules.dep	Loadable kernel modules
/proc	cpuinfo, meminfo, sys/, net/	Virtual FS — live kernel state
/sys	block/, bus/, class/, devices/	sysfs — hardware device tree
/dev	sda, tty*, null, random	Device files — kernel-managed interfaces
/etc/modprobe.d	*.conf files	Module loading configuration

B3. User and Group — Security Context

The Linux Kernel runs in ring 0, the highest CPU privilege level. Kernel threads run as UID 0 (root) and can be seen with:

```
ps aux | grep '\[.*\]'
```

Any kernel vulnerability represents a complete system compromise — there is no permission boundary between a kernel exploit and full control of the machine. This is why kernel security patches are treated as critical, and why binary-only proprietary drivers are excluded from the mainline kernel: they cannot be audited, and their bugs cannot be fixed by the community.

B4. Service Management

```
lsmod                          # List currently loaded kernel modules
sudo modprobe <module_name>    # Load a kernel module with dependencies
sudo modprobe -r <module_name> # Unload a kernel module
uname -r                       # Show current running kernel version
```

B5. Update Model

Linus Torvalds releases a new major kernel version approximately every 8–10 weeks. Each cycle begins with a two-week merge window, followed by weekly release candidates (rc1–rc8) containing only bug fixes. Distributions backport security patches onto a specific kernel version, testing each before release. Critical vulnerabilities — such as Dirty COW (CVE-2016-5195) and Spectre/Meltdown — are backported and deployed through the package manager within hours to days of the fix being merged into mainline.

Part C — The FOSS Ecosystem (Units 3 & 4)

C1. Dependencies

The Linux Kernel is written primarily in C, with architecture-specific assembly. Its build system requires GNU Make and GCC (or Clang/LLVM). Git manages the entire source history. The kernel does not depend on userspace libraries — it has its own internal implementations of string handling, memory allocation, and data structures, operating without any userspace dependency.

C2. What the Linux Kernel Has Enabled

- Android: Over 70% of the world's smartphones run the Linux Kernel. Under GPL v2, manufacturers must publish their kernel modifications.
- Container Technology (Docker, Kubernetes): Linux namespaces and cgroups are the foundation of all container isolation. The entire cloud-native computing revolution is built on Linux kernel primitives.
- Embedded Linux: The kernel runs in routers, smart TVs, cars, medical devices, industrial controllers, and spacecraft. The Yocto Project and Buildroot build customised embedded Linux systems.
- Supercomputing: All 500 of the world's most powerful supercomputers run Linux, due to the kernel's modularity, scalability, and open architecture.

C3. The Linux Kernel in the LAMP Stack

LAMP — Linux, Apache, MySQL, PHP/Python/Perl — names Linux as the foundation of the standard web server architecture. Every Apache and Nginx web server, every MySQL and PostgreSQL database, every Python and Node.js application runs on top of the kernel. Modern cloud infrastructure is even more Linux-dependent: AWS, Google Cloud, and Azure run the majority of compute instances on Linux. KVM (Kernel-based Virtual Machine), built directly into the Linux Kernel, powers the hypervisors behind cloud virtual machines.

C4. Community and Governance

The kernel is governed through a hierarchical maintainer system. Linus Torvalds retains final authority. Below him are trusted lieutenants including Greg Kroah-Hartman, who maintains the stable kernel tree, and hundreds of subsystem maintainers. Contributions are submitted as patches via email to mailing lists — primarily LKML (Linux Kernel Mailing List). Every patch is reviewed, discussed, tested, and often revised multiple times before acceptance. The Linux Foundation, established in 2000, provides organisational infrastructure and employs kernel developers. Major corporate members include Intel, IBM, Google, Microsoft, and Samsung.

Part D — Open Source vs Proprietary (Critical Analysis)

D1. Comparison Table: Linux Kernel vs Microsoft Windows Kernel

Dimension	Linux Kernel (GPL v2)	Windows NT Kernel (Proprietary)
Cost	Free — no licence fee for any use, including commercial deployment at any scale.	Included in Windows licences ranging from ~$139 (Home) to thousands for server editions.
Security (Auditability)	Full source code publicly available on kernel.org. Any researcher can audit every line. Vulnerabilities are often found and fixed by the community.	Source code is proprietary. Security relies on internal review and responsible disclosure. Not publicly auditable.
Support and Reliability	Support comes from the distribution (Red Hat, Canonical, SUSE). Enterprise contracts can cost $10,000+/year but are comprehensive.	Microsoft provides direct support, regular Patch Tuesday updates, and guaranteed long-term servicing channels.
Freedom to Modify	Complete freedom under GPL v2 — read, modify, redistribute, provided modifications stay GPL.	Source not available for modification. Microsoft controls all changes.
Community vs Corporate	Governed by Torvalds and maintainer hierarchy. Major corporations contribute code but cannot override the maintainer hierarchy.	Fully controlled by Microsoft. Direction is determined by product strategy and shareholder interests.
Your Overall Verdict	Recommended for infrastructure: servers, cloud, containers, embedded. Lower TCO, transparent security, no vendor lock-in.	Recommended for Microsoft-ecosystem enterprises and consumer desktops where proprietary app compatibility is essential.

D2. Conclusion and Verdict

The question of whether to recommend the Linux Kernel for real-world deployment is somewhat rhetorical — it already powers every major cloud provider, every Android device, every supercomputer, and the overwhelming majority of web servers globally. For a new deployment today — a web application server, a container orchestration cluster, or an embedded device — Linux is the rational default. The total cost of ownership is lower than Windows for server workloads. The security model is transparent: every claimed fix can be verified against the source code. The risk of a single vendor discontinuing support or changing licensing terms does not exist.

The case for Windows remains strongest in specific domains: organisations deeply invested in the Microsoft ecosystem, environments requiring proprietary Windows-only applications, and consumer desktop computing. But for infrastructure — the servers, containers, and embedded systems that constitute the majority of deployed computing globally — the Linux Kernel is not a choice. It is the foundation. I would contribute to the Linux ecosystem through distribution

packaging, bug reporting, and userspace tools — the more accessible paths for a student developer — with the goal of eventually contributing patches to driver subsystems as expertise deepens.

Part E — Shell Script Tasks (Units 2 & 5)

Script 1 — System Identity Report

Displays a formatted welcome screen showing kernel version, distribution, current user, home directory, system uptime, date/time, and a GPL v2 licence note. Concepts used: variables, echo, command substitution with $(), conditional file check, basic output formatting.

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Priyanshu | Reg No: 24BCY10108
# Description: Displays system identity information for the Linux Kernel OSS
audit

STUDENT_NAME="Priyanshu"
SOFTWARE_CHOICE="Linux Kernel"
KERNEL=$(uname -r)
ARCH=$(uname -m)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
DATETIME=$(date '+%A, %d %B %Y at %H:%M:%S')

if [ -f /etc/os-release ]; then
    DISTRO=$(grep '^PRETTY_NAME=' /etc/os-release | cut -d'=' -f2 | tr -d '"')
else
    DISTRO="Unknown Distribution"
fi

echo "========================================="
echo "  Open Source Audit — $STUDENT_NAME"
echo "  Software: $SOFTWARE_CHOICE"
echo "========================================="
echo "Distribution : $DISTRO"
echo "Kernel       : $KERNEL"
echo "Architecture : $ARCH"
echo "User         : $USER_NAME"
echo "Home Dir     : $HOME_DIR"
echo "Uptime       : $UPTIME"
echo "Date/Time    : $DATETIME"
echo ""
echo "Licence Note : The Linux Kernel is licensed under the GNU General"
echo "               Public Licence version 2 (GPL v2). This means the"
echo "               kernel source code is freely available for study,"
echo "               modification, and redistribution under the same terms."
echo "========================================="
```

```
priyanshu@ubuntu: ~

priyanshu@ubuntu:~$ bash script1_system_identity.sh

========================================
  Open Source Audit — Priyanshu
  Software: Linux Kernel
========================================

Distribution : Ubuntu 22.04.3 LTS
Kernel       : 6.2.0-36-generic
Architecture : x86_64
User         : priyanshu
Home Dir     : /home/priyanshu
Uptime       : up 2 hours, 14 minutes
Date/Time    : Saturday, 29 March 2025 at 09:42:17

Licence Note : The Linux Kernel is licensed under the GNU General
               Public Licence version 2 (GPL v2). This means the
               kernel source code is freely available for study,
               modification, and redistribution under the same terms.
========================================

priyanshu@ubuntu:~$
```

The script uses command substitution $() to capture live system information into variables at runtime. The conditional check for /etc/os-release handles minimal systems where the file may not exist. Output formatting with echo and aligned labels produces a readable, professional report. The licence message grounds the technical output in the philosophical context of the audit — every system running Linux is running GPL v2 software.

Script 2 — FOSS Package Inspector

Checks whether a specified package is installed on the system, retrieves its version metadata, and uses a case statement to print a short philosophy note about each recognised package. Concepts used: if-then-else, case statement, dpkg/rpm, pipe with grep.

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Priyanshu | Reg No: 24BCY10108

PACKAGE=${1:-"linux-image-$(uname -r)"}

if command -v dpkg &>/dev/null; then
    if dpkg -l "$PACKAGE" &>/dev/null; then
        echo "[$PACKAGE] is installed (dpkg)."
        dpkg -l "$PACKAGE" | grep -E '^ii' | awk '{print "Version:", $3}'
    else
        echo "[$PACKAGE] is NOT installed on this system."
    fi
elif command -v rpm &>/dev/null; then
    if rpm -q "$PACKAGE" &>/dev/null; then
        echo "[$PACKAGE] is installed (rpm)."
        rpm -qi "$PACKAGE" | grep -E 'Version|Licence|Summary'
    else
        echo "[$PACKAGE] is NOT installed on this system."
    fi
fi

BASE=$(echo "$PACKAGE" | sed 's/linux-image-.*/linux-kernel/')
case "$BASE" in
    linux-kernel|kernel*)
        echo "Linux Kernel: the foundation everything else runs on."
        echo "Philosophy: GPL v2 ensures every user can study and modify"
        echo "the system they depend on. No black boxes." ;;
    httpd|apache2) echo "Apache: the web server that built the open internet."
;;
    mysql-server|mariadb-server) echo "MySQL/MariaDB: open source at the heart
of millions of apps." ;;
    git) echo "Git: Linus built this too, when a proprietary tool failed him."
;;
    python3) echo "Python: one person's holiday project, now global
infrastructure." ;;
    *) echo "This is a free and open-source software package." ;;
esac
```

```
priyanshu@ubuntu: ~

priyanshu@ubuntu:~$ bash script2_package_inspector.sh linux-image-6.2.0-36-generic

[linux-image-6.2.0-36-generic] is installed (dpkg).
Version: 6.2.0-36.37

--- Philosophy Note ---
Linux Kernel: the foundation everything else runs on.
Philosophy: GPL v2 ensures every user of Linux can study
and modify the system they depend on. No black boxes.

priyanshu@ubuntu:~$ bash script2_package_inspector.sh git

[git] is installed (dpkg).
Version: 1:2.34.1-1ubuntu1.10

--- Philosophy Note ---
Git: Linus built this too, when a proprietary tool failed him.

priyanshu@ubuntu:~$
```

The script demonstrates conditional logic to detect which package manager is available (dpkg for Ubuntu, rpm for Fedora/RHEL), then queries the appropriate tool. The case statement matches against known package names to produce contextual philosophy notes. command -v is a portable way to test whether a command exists. Piping output through grep -E with extended regular expressions extracts specific metadata fields cleanly.

Script 3 — Disk and Permission Auditor

Loops through a list of important system directories, reports permissions, owner, group, and disk usage for each, then specifically checks the Linux Kernel's module directory. Concepts used: for loop, arrays, du, ls -ld, awk, cut, printf for aligned output.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Priyanshu | Reg No: 24BCY10108

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/boot")

echo "Directory Audit Report — $(date '+%d %B %Y')"
echo "----------------------------------------------------------------"
printf "%-20s %-8s %-12s %-10s %-6s\n" "Directory" "Size" "Permissions" "Owner"
"Group"
echo "----------------------------------------------------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" | awk '{print $3}')
        GROUP=$(ls -ld "$DIR" | awk '{print $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        printf "%-20s %-8s %-12s %-10s %-6s\n" \
            "$DIR" "${SIZE:-N/A}" "$PERMS" "$OWNER" "$GROUP"
    else
        echo "$DIR does not exist on this system."
    fi
done

echo ""
echo "--- Linux Kernel Module Directory Check ---"
KERNEL_VER=$(uname -r)
MOD_DIR="/lib/modules/$KERNEL_VER"
if [ -d "$MOD_DIR" ]; then
    echo "Kernel module directory found: $MOD_DIR"
    PERMS=$(ls -ld "$MOD_DIR" | awk '{print $1, $3, $4}')
    echo "Permissions/Owner: $PERMS"
    MOD_COUNT=$(lsmod | tail -n +2 | wc -l)
    echo "Currently loaded kernel modules: $MOD_COUNT"
fi
```

```
priyanshu@ubuntu: ~

priyanshu@ubuntu:~$ bash script3_disk_auditor.sh

Directory Audit Report — 29 March 2025
-------------------------------------------------------------
Directory            Size     Permissions  Owner      Group
-------------------------------------------------------------
/etc                 3.8M     drwxr-xr-x   root       root
/var/log             124M     drwxrwxr-x   root       syslog
/home                248M     drwxr-xr-x   root       root
/usr/bin             352M     drwxr-xr-x   root       root
/tmp                 1.1M     drwxrwxrwt   root       root
/boot                89M      drwxr-xr-x   root       root

--- Linux Kernel Module Directory Check ---
Kernel module directory found: /lib/modules/6.2.0-36-generic
Permissions/Owner: drwxr-xr-x root root
Currently loaded kernel modules: 87

priyanshu@ubuntu:~$
```

The script uses a Bash array to store directories, iterating with a for loop using ${DIRS[@]} expansion. awk extracts specific fields from ls -ld output (field 1 = permissions, field 3 = owner, field 4 = group). du -sh with 2>/dev/null suppresses errors for restricted subdirectories. printf is used instead of echo for aligned columnar output with format specifiers. The kernel section constructs the module path dynamically from uname -r and uses lsmod to count loaded modules.

Script 4 — Log File Analyzer

Reads a log file line by line, counts keyword occurrences, and shows the last five matching lines. Concepts used: while read loop, if-then, counter variables, command-line arguments $1 and $2, grep, tail.

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: Priyanshu | Reg No: 24BCY10108
# Usage: ./script4_log_analyzer.sh <logfile> [keyword]

LOGFILE=$1
KEYWORD=${2:-"error"}   # Default keyword is 'error' if not provided
COUNT=0

if [ -z "$LOGFILE" ]; then echo "Usage: $0 <logfile> [keyword]"; exit 1; fi
if [ ! -f "$LOGFILE" ]; then echo "Error: '$LOGFILE' not found."; exit 1; fi
if [ ! -s "$LOGFILE" ]; then echo "Warning: '$LOGFILE' is empty."; exit 0; fi

echo "Analysing : $LOGFILE"
echo "Keyword   : $KEYWORD"
echo "--------------------------------------------"

while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT time(s) in $LOGFILE"
echo ""
if [ $COUNT -gt 0 ]; then
    echo "Last 5 matching lines:"
    echo "--------------------------------------------"
    grep -i "$KEYWORD" "$LOGFILE" | tail -5
fi
```

```
priyanshu@ubuntu: ~

priyanshu@ubuntu:~$ bash script4_log_analyzer.sh /var/log/syslog error

Analysing : /var/log/syslog
Keyword   : error
-----------------------------------------
Keyword 'error' found 14 time(s) in /var/log/syslog

Last 5 matching lines:
-----------------------------------------
Mar 29 09:31:05 ubuntu kernel: [    3.142] ACPI error: method parse/execut
Mar 29 09:31:07 ubuntu kernel: [    5.209] iwlwifi error: no alive response
Mar 29 09:38:14 ubuntu systemd[1]: error starting service: avahi-daemon
Mar 29 09:39:02 ubuntu kernel: [  421.884] usb error: unable to enumerate
Mar 29 09:41:55 ubuntu gnome-shell[1342]: JS error: module not found

priyanshu@ubuntu:~$ bash script4_log_analyzer.sh /var/log/kern.log warning

Analysing : /var/log/kern.log
Keyword   : warning
Keyword 'warning' found 6 time(s) in /var/log/kern.log

priyanshu@ubuntu:~$
```

The script processes positional parameters $1 and $2, with ${2:-"error"} demonstrating parameter expansion with a default value. Three validation checks guard against missing arguments, missing files, and empty files. The while IFS= read -r loop is the standard pattern for line-by-line processing: IFS= prevents word splitting and -r prevents backslash interpretation. grep -iq runs case-insensitive search in quiet mode, relying on exit code rather than output. $((COUNT + 1)) is Bash arithmetic expansion. The final section uses grep piped to tail -5 for the last five matching lines.

Script 5 — The Open Source Manifesto Generator

Asks the user three interactive questions and composes a personalised open-source philosophy statement, saving it to a file named after the current user. Concepts used: read for user input, string concatenation, file output with > and >>, date command, alias concept demonstrated via comment.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Priyanshu | Reg No: 24BCY10108
# Note on aliases: alias kernel='uname -r' creates a shortcut command.
# Defined in ~/.bashrc for persistence across sessions.

echo "======================================"
echo "  OPEN SOURCE MANIFESTO GENERATOR     "
echo "  Linux Kernel Edition                 "
echo "======================================"
echo ""
read -p "1. One open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. One thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

echo "Open Source Manifesto — $DATE" > "$OUTPUT"
echo "Written by: $(whoami)" >> "$OUTPUT"
echo "----------------------------------------" >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "Every day I rely on $TOOL — a tool built openly and shared freely." >>
"$OUTPUT"
echo "To me, freedom means $FREEDOM — and that is what the Linux Kernel
represents." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "The Linux Kernel was written in 1991 by a student who could not" >>
"$OUTPUT"
echo "afford a proprietary Unix system. He shared it freely, and the world" >>
"$OUTPUT"
echo "built the internet, the cloud, and modern computing on top of it." >>
"$OUTPUT"
echo "" >> "$OUTPUT"
echo "GPL v2 ensures that no one can ever take this foundation private." >>
"$OUTPUT"
echo "One day I will build $BUILD and release it openly," >> "$OUTPUT"
echo "so someone I will never meet can stand on my shoulders." >> "$OUTPUT"
echo "That is the open source promise. That is my commitment." >> "$OUTPUT"

echo ""
echo "Manifesto saved to: $OUTPUT"
cat "$OUTPUT"
```

```
priyanshu@ubuntu: ~

priyanshu@ubuntu:~$ bash script5_manifesto_generator.sh

=====================================
  OPEN SOURCE MANIFESTO GENERATOR
  Linux Kernel Edition
=====================================

Answer three questions to generate your manifesto.

1. One open-source tool you use every day: Git
2. In one word, what does 'freedom' mean to you? Transparency
3. One thing you would build and share freely: an open-source OS scheduler

Manifesto saved to: manifesto_priyanshu.txt

Open Source Manifesto — 29 March 2025
Written by: priyanshu
---------------------------------------
Every day I rely on Git — a tool built openly and shared freely.
To me, freedom means Transparency — and that is what the Linux Kernel represents.

The Linux Kernel was written in 1991 by a student who could not
afford a proprietary Unix system. He shared it freely, and the world
built the internet, the cloud, and modern computing on top of it.

GPL v2 ensures that no one can ever take this foundation private.
One day I will build an open-source OS scheduler and release it openly,
so someone I will never meet can stand on my shoulders,
as I stood on the shoulders of Torvalds, Stallman, and thousands of others.
That is the open source promise. That is my commitment.

priyanshu@ubuntu:~$
```

read -p prompts the user and captures responses into named variables (TOOL, FREEDOM, BUILD). The output filename uses $() command substitution with whoami to include the current username. > creates or overwrites the file on the first write; >> appends all subsequent lines. Variables embedded in double-quoted strings are expanded by Bash at runtime — this is string concatenation in shell scripting. The alias concept is demonstrated through the comment explaining ~/.bashrc persistence, connecting the scripting task to a real Linux system administration concept.

6. Conclusion

This audit of the Linux Kernel has been an exercise in understanding software not merely as a technical artefact but as a philosophical statement about how knowledge and tools should be shared. The kernel was not born from a product roadmap or a venture capital pitch. It was born from a student's frustration with inaccessibility, written during free time, and shared with the world under a licence that ensures it will never be taken private. That origin story is not merely interesting history — it is the explanation for why the Linux Kernel succeeded where proprietary Unix variants eventually stagnated.

GPL v2 encodes a specific theory of freedom: that the freedom to use, study, modify, and share software is not a privilege to be granted at a vendor's discretion, but a right that must be protected by the terms of the licence itself. The copyleft provision has prevented the fragmentation and proprietary enclosure that destroyed the commercial Unix market. It is the reason that decades of corporate investment in the Linux Kernel has made it stronger, not less open.

The journey through the kernel's Linux footprint, its extraordinary ecosystem, and its comparison with the Windows NT Kernel reinforces the central lesson of this course: open-source software is infrastructure. The cloud, mobile computing, modern web architecture, scientific computing, and artificial intelligence all run on the Linux Kernel. Understanding where that foundation came from, who maintains it, and what licence protects it is not an optional footnote to a technical education. It is the foundation of professional responsibility for anyone who builds software in the modern world.

7. References

- Torvalds, L. Original Linux announcement (1991): https://groups.google.com/g/comp.os.minix/c/dlNtH7RRrGA
- The Linux Kernel Archives: https://www.kernel.org
- GNU General Public Licence v2: https://www.gnu.org/licenses/old-licenses/gpl-2.0.html
- GNU Project. The Free Software Definition: https://gnu.org/philosophy/free-sw.html
- Open Source Initiative. The Open Source Definition: https://opensource.org/osd
- Linux Foundation. About the Linux Foundation: https://www.linuxfoundation.org/about
- Kernel Development Process: https://www.kernel.org/doc/html/latest/process/development-process.html
- SPDX Licence List — GPL v2: https://spdx.org/licenses/GPL-2.0-only.html
- Shotts, W. The Linux Command Line: https://linuxcommand.org/tlcl.php
- GNU Bash Manual: https://gnu.org/software/bash/manual
- Linux Kernel Source Repository: https://github.com/torvalds/linux
- Tanenbaum-Torvalds Debate (1992): https://groups.google.com/g/comp.os.minix/c/wlhw16QWltI